CSCI611_Spring26_report

Report summarizing your implementation and findings for both parts

Summary from Part1

In part1, the building image was filtered by sliding the kernel over an image, performing element-wise multiplications and summing the results to produce a new pixel value. This process helped extract features such as edges, textures, and patterns by emphasizing specific spatial structures in the image. Features extracted were:

- Vertical features
- Horizontal Features
- Corners
    - Top right
    - Top left
    - Bottom right
    - Bottom left

Also, various filter mechanisms were used such as blurring and scaling on the blurred images.

Additionally, I also used the California Bald Eagle and put the image through a series of filters.

Part2

Summary from Part2

In part2, an implementation was done to build and train a CNN to classify images from the CIFAR-10 database. The Convolutional Neural Network was built using Pytorch API. The following features were used:

- nn.Conv2d(): for convolution
- nn.MaxPool2d(): for maxpooling (spatial resolution reduction)
- nn.Linear(): for last 1 or 2 layers of fully connected layer before the output layer.
- nn.Dropout(): optional, dropout can be used to avoid overfitting.
- F.relu(): Use ReLU as the activation function for all the hidden layers

The model was initially created with an epoch of 5. However, it was found on that the result was not in the category of “good” (i.e. 70%) for CNN. The initial Test accuracy result was 59%. The optimizer was changed to Adam and the epoch was changed to 8 resulting in accuracy of 72%. Changing the epoch value to 20 resulted in accuracy of 73%. The reason could be due to more shapes in certain images versus the rest. It was found that running more epochs will train the dataset better for the desired results. Also,

using Adam reduced the loss. There are additional parameters that can be changed such as the Fully connected values.

CNN Architecture

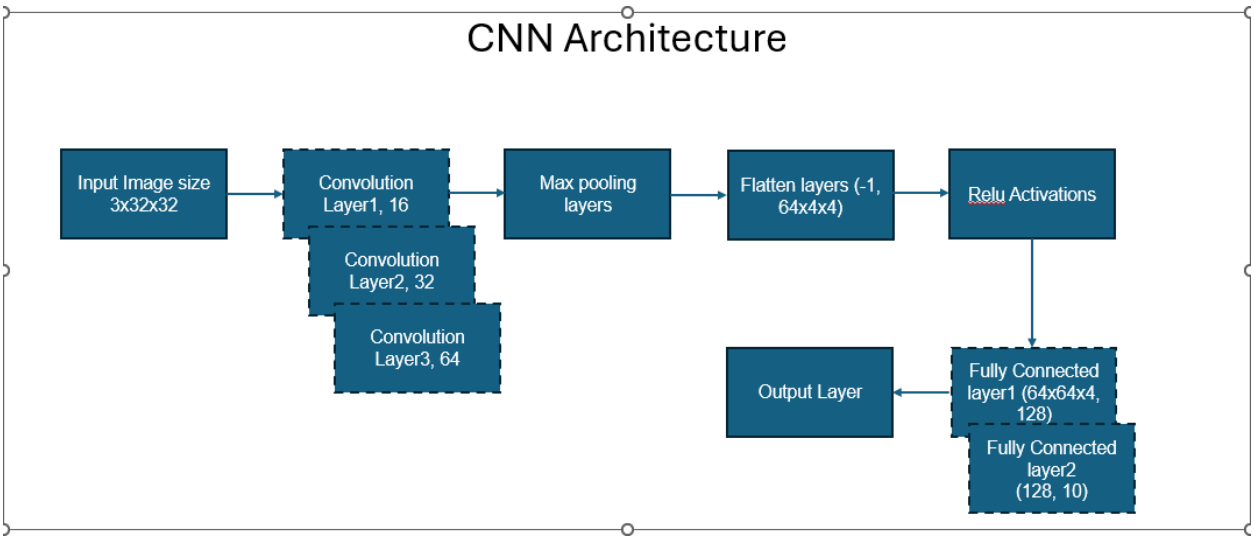

Figure1: CNN Architecture Diagram

Model –

```
Net(
  (conv1): Conv2d(3, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (conv2): Conv2d(16, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (conv3): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
  (fc1): Linear(in_features=1024, out_features=128, bias=True)
  (fc2): Linear(in_features=128, out_features=10, bias=True)
  (dropout): Dropout(p=0.25, inplace=False)
)
```

Training and Loss Function:

Categorical CrossEntropy

SGD – Started with SGD(Training1)

Adam – Switched to Adam(Training2)

Training1

With SGD

Epoch: 1 Training Loss: 2.111371 Validation Loss: 1.825685

Validation loss decreased (inf --> 1.825685). Saving model ...

Epoch: 2 Training Loss: 1.634721 Validation Loss: 1.478532

Validation loss decreased (1.825685 --> 1.478532). Saving model ...

Epoch: 3 Training Loss: 1.440286 Validation Loss: 1.330840

Validation loss decreased (1.478532 --> 1.330840). Saving model ...

Epoch: 4 Training Loss: 1.313348 Validation Loss: 1.254536

Validation loss decreased (1.330840 --> 1.254536). Saving model ...

Epoch: 5 Training Loss: 1.212943 Validation Loss: 1.173161

Validation loss decreased (1.254536 --> 1.173161). Saving model ...

Training2 – Notebook saved with this value and run

Epoch 20 – 937 secs with Adam

Epoch: 1 Training Loss: 1.478695 Validation Loss: 1.211734

Validation loss decreased (inf --> 1.211734). Saving model ...

Epoch: 2 Training Loss: 1.137531 Validation Loss: 1.039157

Validation loss decreased (1.211734 --> 1.039157). Saving model ...

Epoch: 3 Training Loss: 0.988322 Validation Loss: 0.929402

Validation loss decreased (1.039157 --> 0.929402). Saving model ...

Epoch: 4 Training Loss: 0.897569 Validation Loss: 0.857722

Validation loss decreased (0.929402 --> 0.857722). Saving model ...

Epoch: 5 Training Loss: 0.827406 Validation Loss: 0.844335

Validation loss decreased (0.857722 --> 0.844335). Saving model ...

```
Epoch: 6	Training Loss: 0.773503	Validation Loss: 0.826220
Validation loss decreased (0.844335 --> 0.826220).  Saving model ...
Epoch: 7	Training Loss: 0.721014	Validation Loss: 0.808638
Validation loss decreased (0.826220 --> 0.808638).  Saving model ...
Epoch: 8	Training Loss: 0.679287	Validation Loss: 0.795781
Validation loss decreased (0.808638 --> 0.795781).  Saving model ...
Epoch: 9	Training Loss: 0.642177	Validation Loss: 0.800075
Epoch: 10	Training Loss: 0.617892	Validation Loss: 0.788807
Validation loss decreased (0.795781 --> 0.788807).  Saving model ...
Epoch: 11	Training Loss: 0.587468	Validation Loss: 0.791794
Epoch: 12	Training Loss: 0.561568	Validation Loss: 0.807275
Epoch: 13	Training Loss: 0.542979	Validation Loss: 0.840620
Epoch: 14	Training Loss: 0.521481	Validation Loss: 0.791731
Epoch: 15	Training Loss: 0.503787	Validation Loss: 0.832645
Epoch: 16	Training Loss: 0.484242	Validation Loss: 0.864085
Epoch: 17	Training Loss: 0.466054	Validation Loss: 0.839745
Epoch: 18	Training Loss: 0.455418	Validation Loss: 0.837482
Epoch: 19	Training Loss: 0.444017	Validation Loss: 0.846044
Epoch: 20	Training Loss: 0.432254	Validation Loss: 0.884792
```

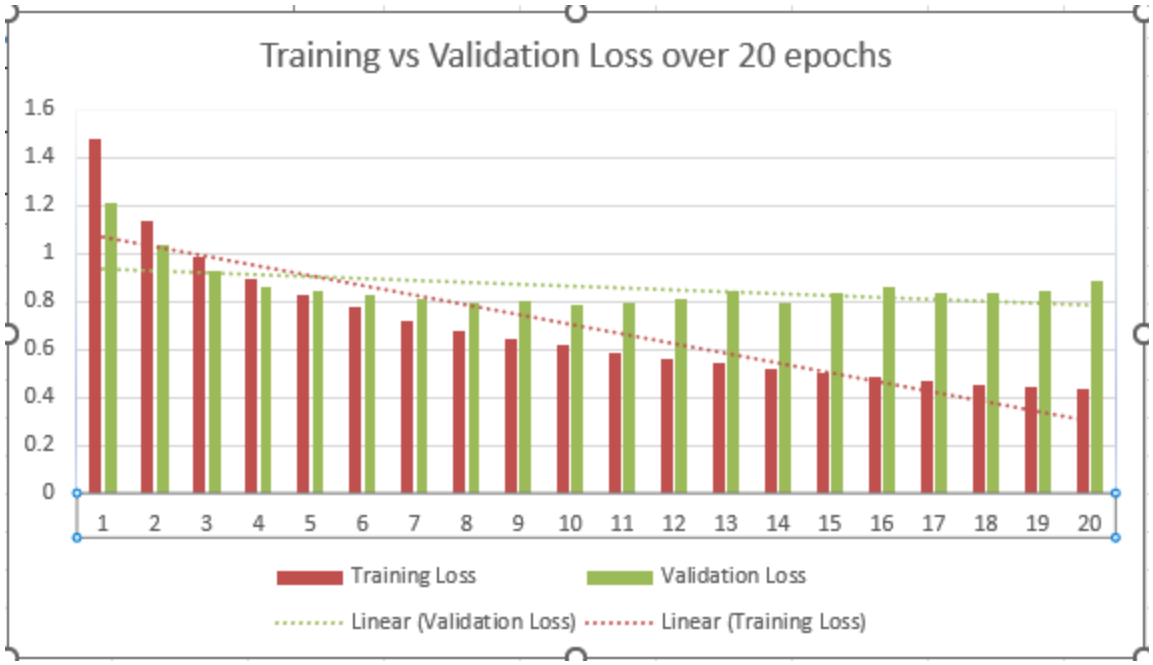

Figure2: Training and Validation Loss over 20 Epochs

Training3:

Epoch: 1 Training Loss: 1.472747 Validation Loss: 1.206344

Validation loss decreased (inf --> 1.206344). Saving model ...

Epoch: 2 Training Loss: 1.123092 Validation Loss: 1.006549

Validation loss decreased (1.206344 --> 1.006549). Saving model ...

Epoch: 3 Training Loss: 0.977851 Validation Loss: 0.902905

Validation loss decreased (1.006549 --> 0.902905). Saving model ...

Epoch: 4 Training Loss: 0.886188 Validation Loss: 0.841065

Validation loss decreased (0.902905 --> 0.841065). Saving model ...

Epoch: 5 Training Loss: 0.810016 Validation Loss: 0.802741

Validation loss decreased (0.841065 --> 0.802741). Saving model ...

Epoch: 6 Training Loss: 0.755611 Validation Loss: 0.801037

Validation loss decreased (0.802741 --> 0.801037). Saving model ...

Epoch: 7 Training Loss: 0.709726 Validation Loss: 0.761964

Validation loss decreased (0.801037 --> 0.761964). Saving model ...

Epoch: 8 Training Loss: 0.673478 Validation Loss: 0.773482

Test the Trained Network

With SGD and Epoch 5

Test Loss: 1.157585

Test Accuracy of airplane: 66% (667/1000)

Test Accuracy of automobile: 84% (843/1000)

Test Accuracy of bird: 54% (549/1000)

Test Accuracy of cat: 30% (308/1000)

Test Accuracy of deer: 41% (416/1000)

Test Accuracy of dog: 58% (582/1000)

Test Accuracy of frog: 63% (630/1000)

Test Accuracy of horse: 57% (572/1000)

Test Accuracy of ship: 75% (755/1000)

Test Accuracy of truck: 58% (582/1000)

Test Accuracy (Overall): **59% (5904/10000)**

With reduced Epoch to 8 and Adam

Test Loss: 0.795950

Test Accuracy of airplane: 78% (789/1000)

Test Accuracy of automobile: 87% (874/1000)

Test Accuracy of bird: 60% (606/1000)

Test Accuracy of cat: 54% (546/1000)

Test Accuracy of deer: 70% (704/1000)

Test Accuracy of dog: 44% (449/1000)

Test Accuracy of frog: 81% (814/1000)

Test Accuracy of horse: 78% (785/1000)

Test Accuracy of ship: 85% (856/1000)

Test Accuracy of truck: 82% (820/1000)

Test Accuracy (Overall): 72% (7243/10000)

<u>With Changes Epoch 20 and Adam</u>

Test Loss: 0.813225

Test Accuracy of airplane: 80% (803/1000)

Test Accuracy of automobile: 85% (850/1000)

Test Accuracy of bird: 47% (474/1000)

Test Accuracy of cat: 53% (538/1000)

Test Accuracy of deer: 74% (740/1000)

Test Accuracy of dog: 69% (690/1000)

Test Accuracy of frog: 83% (834/1000)

Test Accuracy of horse: 69% (698/1000)

Test Accuracy of ship: 83% (836/1000)

Test Accuracy of truck: 83% (838/1000)

Test Accuracy (Overall): 73% (7301/10000)

Sample results with Epoch 8 and Adam

Sample results with Epoch 20 and Adam

Accuracy - ***73% with Epoch 20 and 72% with Epoch 8***

Time taken for training – 937 secs with epoch 20

Number of parameters –

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 16, 32, 32]	448
MaxPool2d-2	[-1, 16, 16, 16]	0
Conv2d-3	[-1, 32, 16, 16]	4,640
MaxPool2d-4	[-1, 32, 8, 8]	0
Conv2d-5	[-1, 64, 8, 8]	18,496
MaxPool2d-6	[-1, 64, 4, 4]	0
Dropout-7	[-1, 1024]	0

```
        Linear-8                [-1, 128]        131,200
        Linear-9                 [-1, 10]          1,290
================================================================
```

Total params: 156,074

Trainable params: 156,074

Non-trainable params: 0

Input size (MB): 0.01

Forward/backward pass size (MB): 0.28

Params size (MB): 0.60

Estimated Total Size (MB): 0.89

What helps and what does not? – Changing these two parameters below helped to move from 59% accuracy to 72% and then 73% accuracy.

1. Increasing the Epoch to 8 and then 20.
2. Also changing the optimizer to Adam helped